

The Token Importance Unit: A Hardware-Verified H2O Heavy-Hitter Eviction Core for KV-Cache Budgeting

Chaithu Talasila

<https://github.com/LonghornSilicon/token-importance-unit>

Abstract

The key-value (KV) cache of an autoregressive transformer grows linearly with context, and on an on-die inference accelerator it must be held within a fixed SRAM budget. We present the Token Importance Unit (TIU), the third block of the LonghornSilicon LLM inference accelerator: a hardware eviction core that implements the H2O heavy-hitter policy [Zhang et al., 2023] — retain a recent local window plus the tokens with the largest accumulated post-softmax attention mass, evict the rest. We first validate the algorithm on Qwen2: on long context (WikiText-2, 1024-token windows) a 25% KV budget with a 50/50 recency/heavy-hitter split is near-lossless (+1.0% perplexity), and an 8-bit fixed-point accumulator is loss-free for the eviction ranking ($\Delta -0.002$ HellaSwag acc.norm). We then carry the policy to silicon. The RTL core is deliberately minimal — distributed per-slot saturating accumulators, a serialized-argmin victim scan, and a per-slot keep/demote tier comparator — and passes 29/29 directed self-checking tests and 40/40 eviction checks replayed from real Qwen2 attention traces, bit-exactly matched by a Python reference model. Through the LibreLane/OpenROAD open flow on SkyWater Sky130A the design signs off with *zero* violations of any kind (setup WNS +16.6 ns, hold WNS +0.18 ns) in a 13,833 μm^2 die at $\sim 0.58 \mu\text{W}$. Composed with the two sibling blocks (KV Cache Engine, Adaptive Precision Attention) on one Qwen2 attention, the full stack — 75% eviction $\times$ 4-bit KV $\times$ INT8 compute — costs only $\Delta -0.031$ HellaSwag acc.norm versus FP16, with the precision controller 99.99% INT8.

appends a key and a value vector per layer per head, so the cache grows linearly with sequence length while the model weights stay fixed. On a custom accelerator with a bounded on-die SRAM budget, something must decide *which* cached tokens to keep as context outgrows the budget.

The Heavy-Hitter Oracle (H2O) [Zhang et al., 2023] is the canonical answer: a small, stable set of tokens receives most of the attention mass across the whole sequence, so tracking each token’s accumulated post-softmax attention and evicting the least-attended tokens preserves accuracy at a fraction of the cache. H2O is prior art *as an algorithm*. The contribution of this work is the streaming *silicon* that implements it, its bit-accurate software model for compiler co-design, and its integration with a mixed-precision KV codec through a keep/demote/evict tier signal.

We follow the same discovery-to-silicon path as the sibling Adaptive Precision Attention block: validate the policy and its fixed-point sizing on real language models, reduce it to a small closed-form-FF datapath, verify it in RTL against a bit-exact reference, and sign it off end-to-end through an open ASIC flow.

Contributions.

- An accuracy-versus-budget validation of H2O on Qwen2, establishing a gold operating point (25% KV budget, 50/50 recency/heavy-hitter split) and the fixed-point accumulator width (8 bits, loss-free).
- A minimal eviction-core RTL — distributed saturating accumulators, a serialized-argmin victim scan, and a per-slot tier comparator — with a closed-form flip-flop count, verified 29/29 directed and 40/40 on real Qwen2 replay traces, and a bit-exact Python reference model.
- A zero-violation Sky130 sign-off and a 16FFC

1 Introduction

The attention KV cache is the dominant memory cost of long-context LLM inference: every generated token

projection.

- A three-block integration measurement (TIU + KV Cache Engine + Adaptive Precision Attention) showing the full stack holds within $\sim 3\%$ of FP16.

2 Related Work

KV-cache eviction. H2O [Zhang et al., 2023] retains heavy hitters by accumulated attention mass; our eviction datapath is its hardware realization. The complementary axis is *compression* rather than eviction: KIVI [Liu et al., 2024] quantizes keys per-channel and values per-token to 2 bits, and KVQuant [Hooper et al., 2024] pushes to sub-4-bit with per-channel key quantization and outlier isolation for very long context. The LonghornSilicon KV Cache Engine (block 2) uses a ChannelQuant codec in the same family; the TIU’s tier signal drives its per-token value precision (§6), so eviction and compression compose.

Attention compute. FlashAttention [Dao et al., 2022] established the tiled, IO-aware attention computation that the accelerator’s Attention Compute Unit targets; the post-softmax attention mass the TIU accumulates is a by-product of that same scan.

3 Algorithm Validation

We validate H2O on Qwen2-0.5B before committing the policy to RTL. The signal — post-softmax attention mass — was settled by the ACU sparsity study (post-softmax mass predicts token importance at $r \approx 0.99$; pre-softmax proxies at $r \approx 0$). Here we size the *policy*: the KV budget, the recency split, per-head vs. shared selection, and the accumulator bit-width.

3.1 Accuracy vs. KV budget

We apply H2O eviction to every (layer, head) of Qwen2-0.5B and measure HellaSwag `acc_norm` as a function of the cache budget C (a fraction of the sequence length), with a recent local window of $L = C/2$ tokens plus the top $C - L$ heavy hitters (Table 1). Accuracy is near-lossless ($|\Delta| \leq 0.006$) down to a 25% budget, then bends sharply below $\sim 15\%$.

3.2 Long-context knee

HellaSwag sequences are short, so eviction barely triggers there (kept fraction ≈ 0.97 at a 25% budget) and

Table 1: HellaSwag `acc_norm` vs. KV budget, Qwen2-0.5B, $n=500$, recent-window share $L/C = 0.5$.

KV budget	<code>acc_norm</code>	Δ vs. full
1.00	0.498	—
0.75	0.490	−0.008
0.50	0.496	−0.002
0.35	0.496	−0.002
0.25 (gold)	0.492	−0.006
0.15	0.454	−0.044
0.10	0.376	−0.122

the knee is soft. Concatenating WikiText-2 into non-overlapping 1024-token windows (131,072 tokens) exercises the budget properly: the kept fraction drops to 0.44 at a 25% budget, and perplexity is flat to 50% (+0.0%), near-flat at 25% (+1.0%), then bends — +4.0% at 15%, +8.9% at 10%, +25.7% at 5%. On genuinely long context the 25% gold budget is near-lossless and the knee sits around 10–15%, sharper and cleaner than HellaSwag showed.

3.3 Per-head vs. shared budget

A shared budget (rank tokens by attention mass summed over heads, one keep-set per layer) is cheaper silicon — $1/H$ the accumulator width and a single eviction index — but heads disagree on which tokens matter. At the gold 25% budget a shared keep-set costs −0.040 `acc_norm` versus per-head, well past the ± 0.02 per-block gate; the gap only closes at loose ($\geq 50\%$) budgets where there is no eviction pressure. We therefore keep **per-head** selection in the RTL.

3.4 Accumulator bit-width

The RTL stores each token’s accumulated mass in a fixed-point accumulator of `SCORE_WIDTH` bits (unsigned, round-to-nearest, saturating), applied before the heavy-hitter selection. Table 2 sweeps the width at the gold budget. Accuracy is flat down to **8 bits** ($\Delta -0.002$ vs. an fp32 accumulator), falls off a cliff at 6 bits (−0.022), and is broken at 4. We set `SCORE_WIDTH` = 8: loss-free with margin, half the storage of a naive 16-bit register, and a smaller argmin read-mux (which later drove the Sky130 max-transition closure).

Table 2: HellaSwag acc_norm vs. accumulator width, gold budget, Qwen2-0.5B, $n=500$. fp32 accumulator reference = 0.492.

SCORE_WIDTH	acc_norm	Δ vs. fp32
16	0.498	+0.006
12	0.496	+0.004
10	0.498	+0.006
8	0.490	-0.002
6	0.470	-0.022
4	0.456	-0.036

4 RTL Design and Verification

4.1 Datapath

The DUT (`rtl/token_importance_unit.sv`) maintains `N_SLOTS` KV-cache slots, each a (`score`, `valid`) pair, and implements three operations plus a combinational tier read:

```

LOAD( $s$ ):  $\text{score}[s] \leftarrow 0, \text{valid}[s] \leftarrow 1$ 
ACC( $s, w$ ):  $\text{score}[s] \leftarrow \text{sat\_add}(\text{score}[s], w)$ 
EVICT():  $\rightarrow \arg \min_{k: \text{valid}[k]} \text{score}[k]$ 
TIER( $\theta$ ):  $\text{tier\_keep}[k] = \text{valid}[k] \wedge (\text{score}[k] \geq \theta)$ 

```

Three design choices make it place-and-route at a real clock. First, eviction is a *serialized* argmin (one comparator over `N_SLOTS` + 1 cycles), not a wide combinational min tree. Second, the accumulators are *distributed* — each slot owns a small saturating adder driven only by its own score and the broadcast weight, avoiding a shared adder-result net fanning out to `N_SLOTS` register-input muxes. Third, only the control flip-flops (`state`, `valid[]`, `evict_valid`) are reset; the `score[]` datapath is not (a slot is zeroed by `LOAD` before it is ever read), which keeps `rst_n` fanout at ~ 11 instead of ~ 113 . The flip-flop count is closed-form,

$$\text{FFs} = \text{N_SLOTS} \cdot (\text{SCORE_WIDTH} + 1) + \text{SCORE_WIDTH} + 3 \text{ SLOTS} + 3$$

92 at `N_SLOTS=8` (95 synthesized; yosys leaves ~ 3 slot-index FFs un-merged, the value the CI gate pins).

The *H2O recent-window and budget bookkeeping* is a control-layer wrapper, not part of this core — the silicon is accumulate + argmin + tier, and the software layer decides when to `LOAD` and `EVICT`, exactly as block 1 shipped just the ratio gate.

4.2 Functional verification

Two self-checking testbenches drive the same DUT, each maintaining a shadow model of the exact eviction semantics (serialized argmin, strict-less scan seeded at slot 0):

- **Directed TB** (`tb_token_importance_unit.sv`): directed cases (distinct masses, tie-breaking, saturation past `SCORE_MAX`, tier thresholding, free-on-evict) plus a 200-op randomized soak. Result: 29/29 pass.
- **Replay TB** (`tb_realdata.sv`): 444 `LOAD/ACC/EVICT` ops generated from a real Qwen2-0.5B forward pass by `analysis/gen_tiu_testvectors.py` (48 tokens streamed through 8 slots, 40 evictions), each `EVICT` checked against the victim the argmin should produce. Result: 40/40 evictions bit-exact.

4.3 Bit-exact reference model

A Python model (`sw/reference_model/tiu_ref.py`) mirrors the RTL op-for-op — the same distributed saturating accumulators, the same serialized argmin seeded at slot 0, the same tier comparator. Replaying the committed golden trace through both the model and the SystemVerilog DUT yields identical victims on all 40 evictions (`test_tiu_ref.py`), establishing Python $\leftrightarrow$ RTL parity. The model is the compiler team’s target before silicon or FPGA exists.

5 Sky130 Physical Sign-off

We implement the design end-to-end through the LibreLane/OpenROAD [LibreLane Contributors, 2024] open ASIC flow (synthesis, floorplan, placement, CTS, routing, extraction, multi-corner STA, DRC, LVS, antenna) on SkyWater Sky130A. The design signs off with *zero* violations of any kind (Table 3).

Headline numbers: a 13,833 μm^2 die at $\sim 0.58 \mu\text{W}$, 207,500 FFs, Closing slew and fanout (748 $\rightarrow$ 0 violations) took four RTL/config levers: resetting only the control flip-flops (`rst_n` fanout 113 $\rightarrow$ 11), distributing the per-slot accumulators, dropping `SCORE_WIDTH` from 16 to 8 (§3.4), and signing off at the Sky130 cells’ real `max_transition` of 1.5 ns (the flow’s 0.75 ns default is half the PDK spec). The tier port is emitted as `N_SLOTS` parallel comparators, adding no fanout to the argmin path, so it did not perturb the clean sign-off.

Table 3: Sky130A sign-off, 25 ns clock. Zero violations across every check.

Check	Count
Setup (WNS +16.6 ns)	0
Hold (WNS +0.18 ns)	0
Max transition (slew)	0
Max capacitance	0
Max fanout	0
Routing DRC / Magic DRC	0
LVS errors	0
Antenna violations	0

Physical proxy. The shipped RTL default is $N_SLOTS = 8$ (what the testbench verifies bit-exact). The physical run synthesizes an $N_SLOTS = 4$ proxy via `SYNTH_PARAMETERS` — exactly as the KV Cache Engine ships a `VECTOR_DIM = 8` proxy — to shrink the argmin-mux fanout and clock tree so the clock-root fanout clears the flow’s limit, with the datapath parameter-identical. Flip-flop count: 95 at $N_SLOTS=8$ (real), 53 at $N_SLOTS=4$ (proxy).

16FFC projection. Applying the published Sky130→TSMC 16FFC scaling ratios used in the sibling block’s cross-check ($\sim 5\times$ area reduction, $\sim 2\times$ speed gain, $\sim 10\times$ dynamic-power reduction) to the signed-off Sky130 numbers projects the eviction core to $\sim 2,800 \mu m^2$ and $\sim 0.06 \mu W$ in 16FFC — negligible silicon against the multi-kilobyte KV SRAM it manages. A real Genus / Innovus sign-off on the University Program shuttle is future work; the projection uses documented per-cell ratios, not measured 16FFC values.

6 Three-Block Integration

We compose all three live LonghornSilicon blocks in one custom Qwen2 attention, in chip order: the KV Cache Engine (block 2) decompresses $K/V \rightarrow$ attention scores $\rightarrow$ the TIU (block 3) rules keep/demote/evict $\rightarrow$ Adaptive Precision Attention (block 1) routes the $S \cdot V$ MAC between INT8 and FP16. The TIU emits the tier signal the KVCE already consumes: `evict` drops a token’s K and V; `tier_keep=1` stores its *value* at CQ-8, `tier_keep=0` at CQ-4. The tier is a per-token *value*-precision lever only — per-token key demotion collapses GQA (measured -0.17), so keys stay uniform per-channel.

The full stack (Table 4) lands at ~ -0.03 acc.norm

Table 4: Full-stack HellaSwag acc_norm vs. FP16 full cache, Qwen2, gold TIU config (25% budget, $L/C=0.5$).

Config	0.5B ($n=1000$)	1.5B ($n=1000$)
fp16 (baseline)	0.489	0.590
TIU evict only	-0.016	-0.034
KVCE cq4+ only	-0.015	-0.003
APA only	+0.001	-0.002
ALL 3	-0.033	-0.030
ALL 3 + graded demote	-0.023	-0.029

— the expected cumulative cost of three aggressive, independent optimizations (75% cache eviction $\times$ 4-bit $KV \times INT8$ compute) — with APA effectively free (99.99% INT8, $\approx FP16$). Graded *value* demotion (spend bits on heavy hitters, starve the rest) recovers ~ 1 point on the 0.5B model. Using the exact RTL `tier_keep` semantics (2-tier keep→CQ-8 / demote→CQ-4 by an accumulated-mass threshold), the tier-handshake configuration measures $\Delta -0.031$ at a 25% budget with APA fully active. To hold under a ± 0.02 combined envelope, the TIU budget backs off to $\sim 35\%$ (where TIU alone is ≈ -0.006); 25% is the aggressive operating point.

7 Conclusion

The Token Importance Unit carries the H2O heavy-hitter policy from algorithm to signed-off silicon. On Qwen2 a 25% KV budget with a 50/50 recency split is near-lossless ($+1.0\%$ long-context perplexity) and an 8-bit accumulator is loss-free. The eviction core — distributed saturating accumulators, a serialized-argmin victim scan, and a per-slot tier comparator — passes 29/29 directed and 40/40 real-Qwen2 replay checks, is bit-exactly reproduced by a Python reference model, and signs off through the LibreLane/OpenROAD open flow on Sky130 with zero violations in a $13,833 \mu m^2$ die at $\sim 0.58 \mu W$, projecting to $\sim 2,800 \mu m^2$ in TSMC 16FFC. Composed with the KV Cache Engine and Adaptive Precision Attention, the full three-block stack holds within $\sim 3\%$ of FP16 — an eviction core ready for the University Program shuttle.

References

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Ad-*

vances in Neural Information Processing Systems, volume 35, pages 16344–16359, 2022.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems*, 2024.

LibreLane Contributors. LibreLane: An open-source asic implementation flow built on OpenROAD. <https://github.com/librelane/librelane>, 2024.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning*, 2024.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.